

CAP776

Programming in Python



Lecture #9

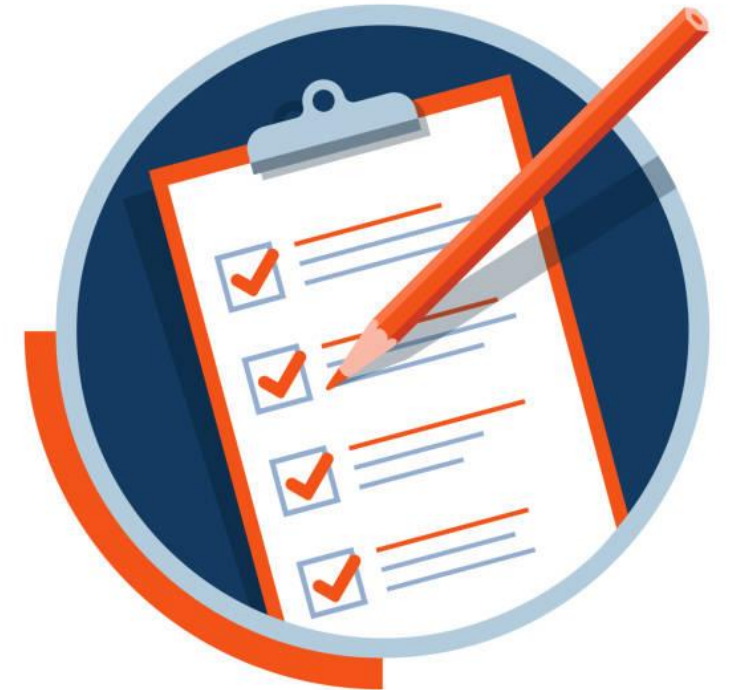
File Read, Write and Handling + **Minor Project #1**

Recap

- **Type conversion**
 - Implicit
 - Explicit
- **Coercion**
- **Strings**
 - Slicing
 - Stripping

Today's Agenda

- File
- File Handling
 - Built-in functions
 - open
 - read
 - write
 - append
 - openpyxl library
- **Minor Project #1 - Allocation**



Transient & Persistent Program

- **Transient**

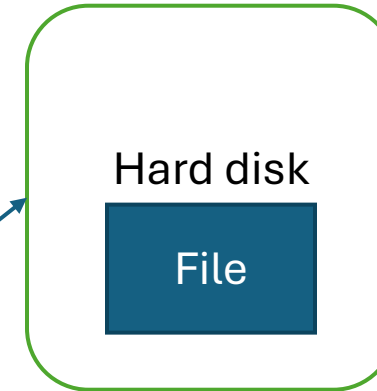
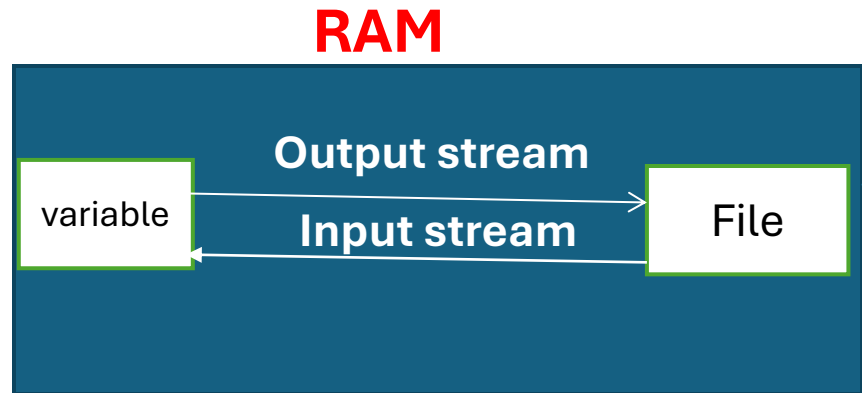
- The programs run for a short period of time to provide on-spot output
- Data is volatile and is stored in primary/temporary memory

- **Persistent**

- The programs run for a long duration to providing anytime output
- Data is permanent and is stored in secondary/permanent memory



Stream: flow of data



Data from variable to file- output stream
Data from file to variable-input stream

We have predefine functions to manage all these things

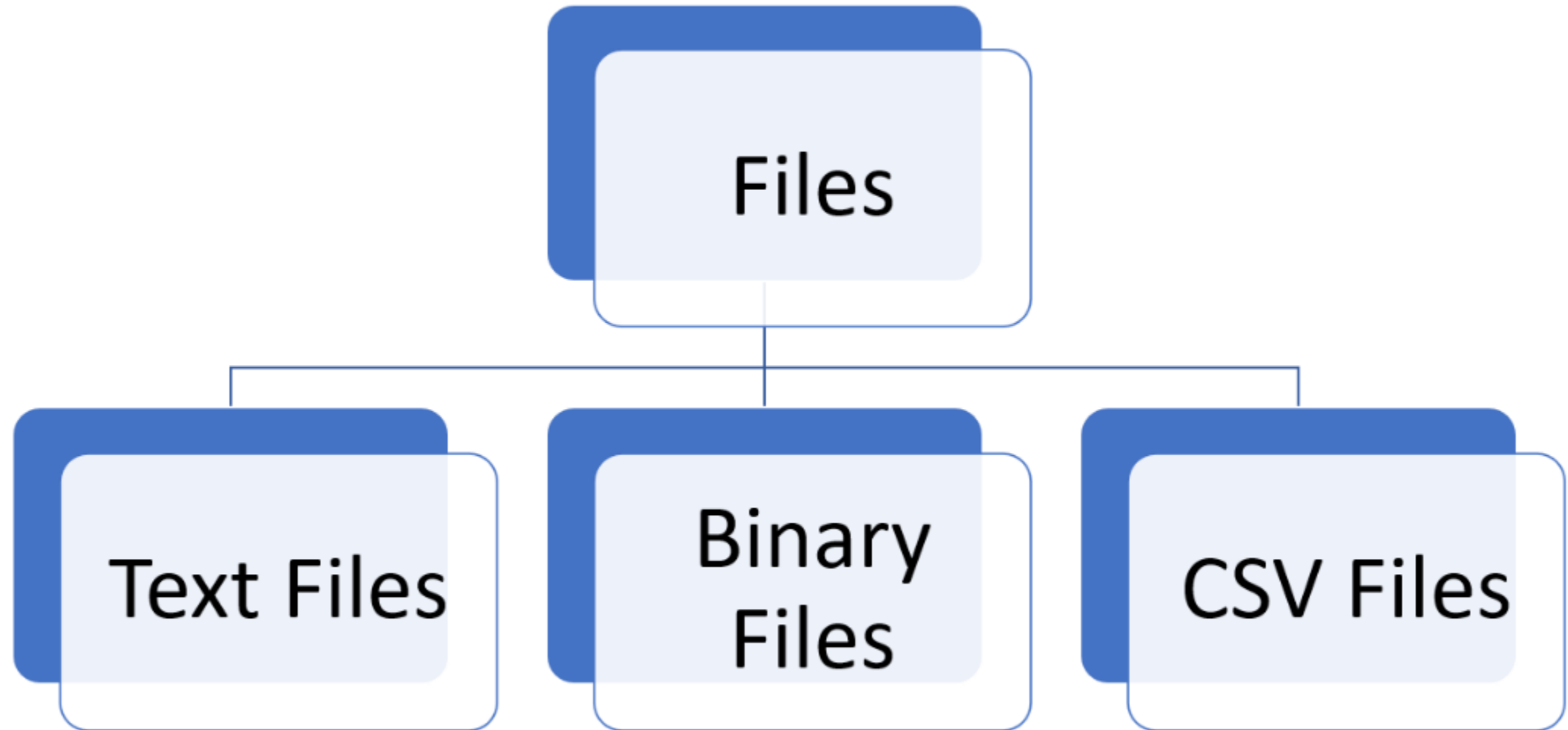
Keeping record of products:

➤ using file handling mechanism

 Good item name \$1280 - \$1980	 The name of product \$280	 Good item name \$280	 Good item name \$280
 Good item name \$1280 - \$1980	 The name of product \$280	 The name of product \$280	 The name of product \$280
 The name of product \$1280 - \$1980	 The name of product \$280	 The name of product \$280	 The name of product \$280

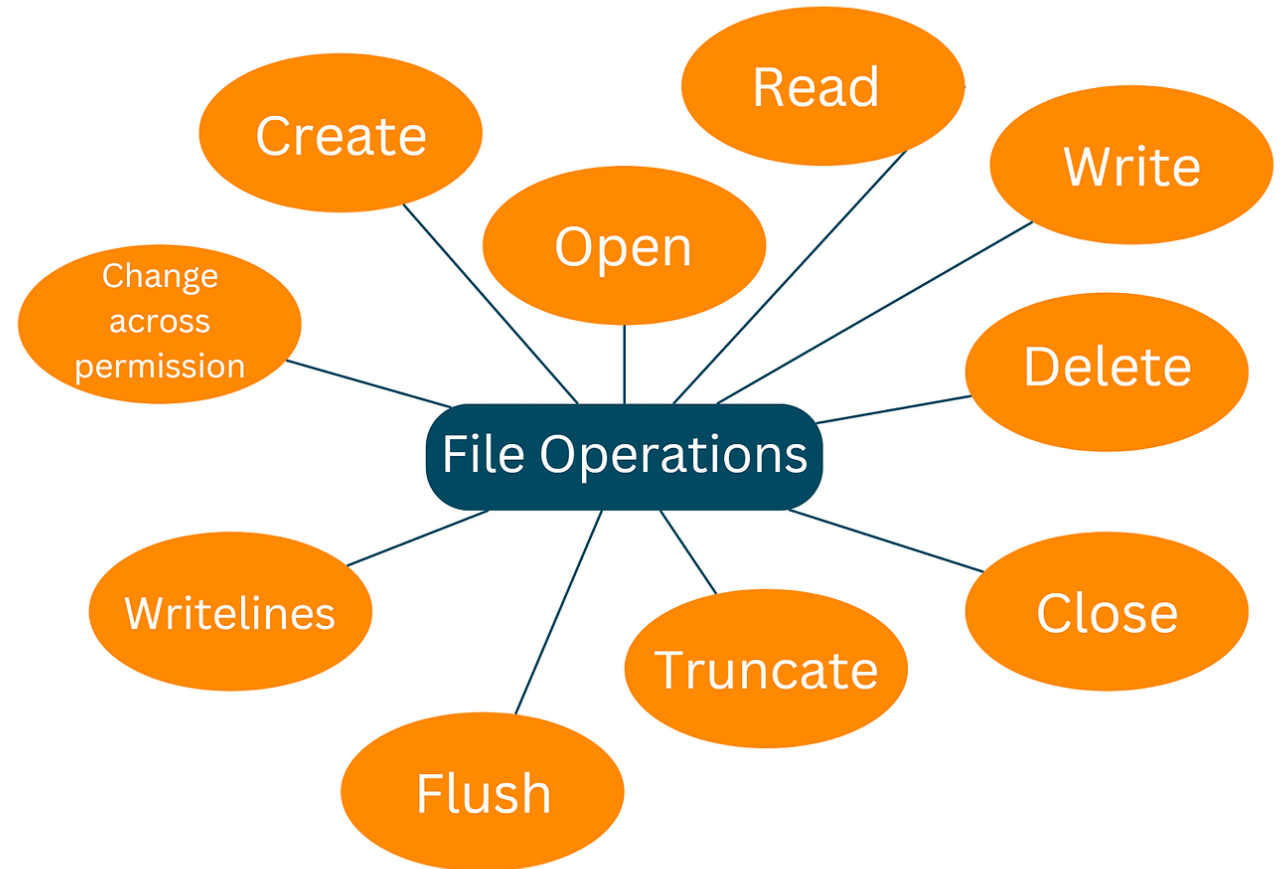


Files



File Handling in Python

- File handling in python that includes file operations like creating, reading, writing and deleting.
- Python has various built-in functions like `open()`, `read()` and `write()` with different modes: `r`(read), `w`(write) and `a`(append)



Built-in Functions for File Handling

- **open():**

Syntax: file_object = open('filename', 'mode')

Different File Modes:

'r' (read): Opens the file for reading. An error occurs if the file does not exist.

'w' (write): Opens the file for writing. If the file exists, it overwrites the file. If the file does not exist, it creates a new one.

'a' (append): Opens the file for appending. If the file does not exist, it creates a new one.

'r+' (read and write): Opens the file for both reading and writing. An error occurs if the file does not exist.

'w+' (write and read): Opens the file for reading and writing. If the file exists, it overwrites the file. If the file does not exist, it creates a new one.

'a+' (append and read): Opens the file for appending and reading. If the file does not exist, it creates a new one.

'b' (binary): Used for binary files (like images or videos) in combination with other modes (e.g., 'rb', 'wb').

Define a function register to append details

```
def register():  
    print("User Registration")  
    name = input("Enter your name: ")  
    email = input("Enter your email: ")  
    password = input("Enter your password: ")  
    file=open("login.csv",'a')  
    file.write(f"{name},{email},{password}\n")  
    file.close()  
    print("Registration Done")
```

Reading an Excel file

[openpyxl](#) is a Python library used to read, write, and manipulate modern Excel spreadsheets (.xlsx, .xlsm, .xltx, and .xltm files) without needing Microsoft Excel installed

```
pip install openpyxl # Install openpyxl
```

```
import openpyxl # Python library for Excel operations
```

```
wb = openpyxl.load_workbook(filename, data_only=True)
```

```
ws = wb[sheet_name]
```

Working on an Excel file

A	B	C	D
Name	Age	Height	Weight
L	12	120	40
M	32	140	65
N	23	160	72
O	23	140	46
P	45	130	85

Average: 61.6

```
[40]
[40, 65]
[40, 65, 72]
[40, 65, 72, 46]
[40, 65, 72, 46, 85]
Average: 61.6
```

```
import openpyxl
import math

def average_column(filename, sheet_name, column):
    wb = openpyxl.load_workbook(filename, data_only=True)
    ws = wb[sheet_name]

    values = []

    for row in range(2, ws.max_row + 1):
        value = ws.cell(row, column).value
        if value is not None:
            values.append(value)

    print(values)

    return math.sum(values) / len(values)

col_average = average_column("CA1.xlsx", "Sheet1", 4)

print("Average:", col_average)
```

Sandbox: Step #1

Create an Excel file to capture your friends' details.

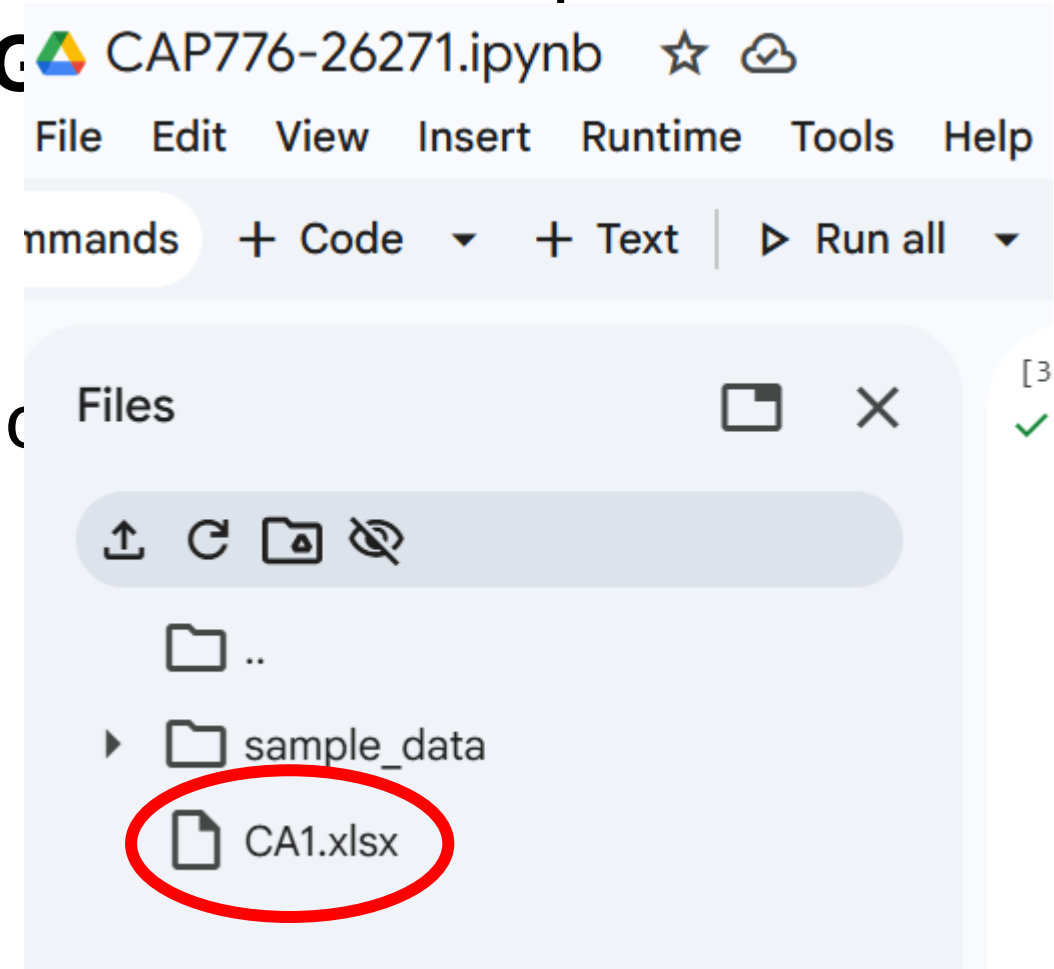
- **Name**
- **Age**
- **Height**
- **Weight**
- **City**
- **State**

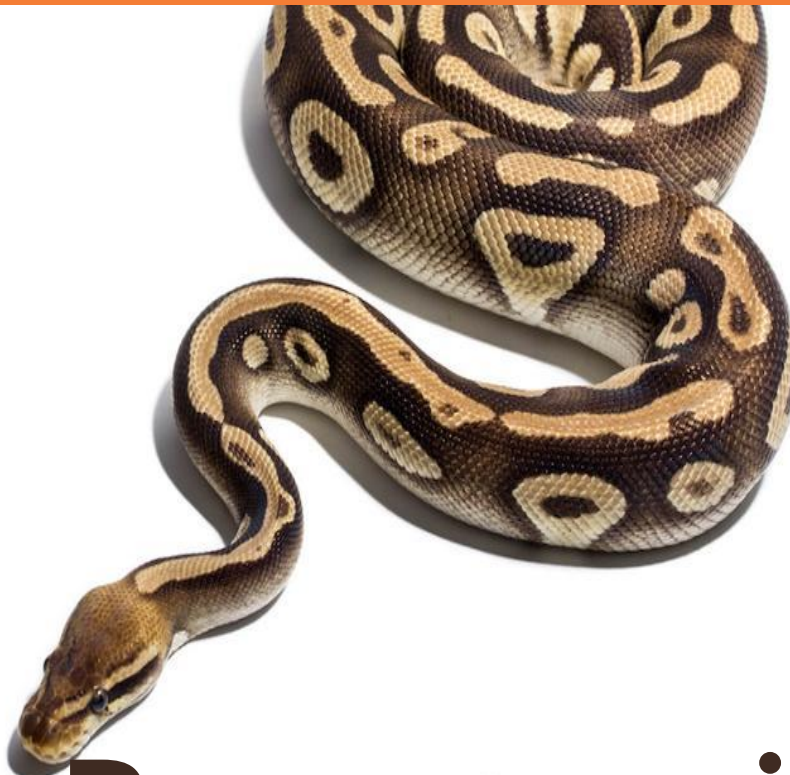
Enter at least **FIVE** records in the file and save it as friends.xlsx

Sandbox: Step #2

Read this file and write functions to assess and report the **AVERAGE HEIGHT, AGE, and WEIGHT**

- Use openpyxl
- Read the file
- Use basic mathematical expressions and functions





CAP776 – Programming in Python

Minor Project #1

Prof. Lovi Raj Gupta, PhD

Pro Vice-Chancellor
Lovely Professional University, Punjab



My Data My Story



Name:		Registration	
Course / Sec			

[illegible]

Parameters

Personal Activity Index	PAI
Tech Productivity	TPI
Academic Activity	AAI
Physical Activity	PhAI
Sleep & Recovery	SRI
Activity Balance	ABI
Time Utilization	TUI
Experience Index	EI
Data Continuity Index	DCI

Tech Productivity (TPI)

$$TPI = \frac{\sum Coding}{Number\ of\ Valid\ Days}$$

min/day

How much time, on average, did I devote to coding?

Academic Activity (AAI)

$$AAI = \frac{\sum (Study + Class)}{Number\ of\ Valid\ Days}$$

min/day

How much time, on average, did I devote to formal and independent academic activity?

Physical Activity (PhAI)

$$PhAI = \frac{\sum Fitness}{Number\ of\ Valid\ Days}$$

min/day

How much time, on average, am I active in a day?

Sleep & Recovery (SRI)

$$SRI = \frac{\sum Sleep}{Number\ of\ Valid\ Days}$$

min/day

My sleep monitoring parameter

Activity Balance (ABI)

$$ABI = \frac{\sum \text{Free/Unaccounted Time}}{\text{Number of Valid Days}}$$

min/day

How much of my day, on average, was not accounted for by the activities I recorded?

Time Utilization (TUI)

$$TUI = \frac{\sum Total Tracked Time}{Number of Valid Days}$$

min/day

The time that was not recorded, on average?

Experience Index (EI)

$$EI = \frac{\sum (Feeling + Satisfaction + Energy)}{3 \times \text{Number of Valid Days}}$$

My overall experience on a scale of 1-5

Data Continuity (DCI)

$$DCI = \frac{\textit{Valid Recorded Days}}{\textit{Expected Days}} \times 100$$

An index to measure the continuum in data recording since 13th Aug 2026.

The last day of the recording will be 21st Sept 2026

Personal Activity Index (PAI)

15%**20%****15%****20%****15%****10%****15%**

$$PAI = 0.15TPI_s + 0.20AAI_s + 0.15PhAI_s + 0.20SRI_s + 0.15TUI_s + 0.10EI_s + 0.05DCI$$

The overall participation and time utilization & distribution parameter

Correlation

- Coding $\leftrightarrow$ Energy
- Sleep $\leftrightarrow$ Energy
- Study $\leftrightarrow$ Satisfaction

Report

- **Furnish the soft copy with entered values and notes**



Rubric

Evaluation Component	Marks	What is Evaluated
Data File & Data Continuity	10	<i>Student's own activity data, correct date sequence, completeness and consistency</i>
File Handling & Data Reading	10	<i>Reading the Excel file correctly using Python; appropriate use of file-handling concepts</i>
Data Validation & Exception Handling	10	<i>Checking missing/invalid values, handling errors using try-except and appropriate validation</i>
Python Fundamentals	10	<i>Variables, data types, operators, conditions, loops and basic calculations</i>
Functions & Modular Programming	15	<i>Use of user-defined functions; logical decomposition; reusable code</i>
Python Data Structures	10	<i>Appropriate use of lists, dictionaries, tuples/sets where applicable</i>
Calculation of Activity Indices	15	<i>Correct calculation of TPI, AAI, PhAI, SRI, ABI, TUI, EI and DCI</i>
Relationship Analysis	10	<i>At least 3 meaningful relationships, e.g. Sleep–Energy, Study–Satisfaction, Coding–Energy</i>
Interpretation & Findings	5	<i>Evidence-based personal findings derived from the student's own data</i>
Documentation & Reproducibility	5	<i>Readability, comments, instructions and ability to reproduce the results</i>
TOTAL	100*	<i>*Scaled down to 15</i>

Important points

- **Do not furnish a report with fake or fabricated data**
- **Do not use NumPy or Pandas; only use the present understanding**
- **Create reusable functions, as they will be used in Minor #2 and the Major Project**

Most Important point

- **If in doubt**
 - **Try it once more**
 - **Explore**
 - **Talk to co-instructors**
 - **Come to me**
- **Do not copy or use LLMs (we will figure out)**



Start date

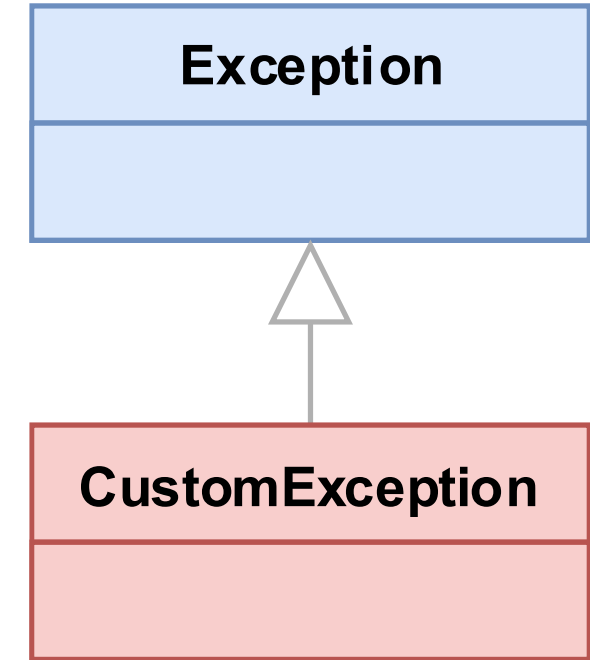
- **15th Sept 2026**

Submission closing

- **28th Sept 2026 00:00 Midnight**

Coming up

- **Exception Handling**
 - Errors & Exceptions
 - Built-in exceptions
 - User defined exceptions
 - Try – Except
 - Try – Except – Else
 - Try – Except – Finally
 - **Raise**
- **JSON**



```
class CustomException(Exception):  
    def __init__(self,msg='Exception occurs'):  
        self.msg = msg  
        super().__init__(self.msg)  
  
try:  
    raise CustomException()  
except CustomException as ex:  
    print(ex)
```

Exception occurs



Up next...

- **Exception Handling**
- **JSON**